Practical 4

Part A:

Aim:

1. Password strength (PasswordChecker.java + Driver.java)
(a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter,
contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or
String methods.
(b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–
1), "Medium" (2–3) or "Strong" (4).
(c) In Driver.main: test several passwords; for each, print which rules passed and the
final label.
Expected output: "abc" → Weak; "Abcd1234!" → Strong.

Code:

PasswordChecker.java

```java
public class PasswordChecker {

    // Check whether password length is at least 8
    public static boolean hasMinimumLength(String pw) {
        return pw.length() >= 8;
    }

    // Check whether password contains an uppercase letter
    public static boolean hasUppercase(String pw) {
        return pw.matches(".*[A-Z].*");
    }

    // Check whether password contains a digit
    public static boolean hasDigit(String pw) {
        return pw.matches(".*[0-9].*");
    }

    // Check whether password contains a special character
    public static boolean hasSpecialCharacter(String pw) {
        return pw.matches(".*[^a-zA-Z0-9].*");
    }

    // Calculate password strength
    public static String strength(String pw) {

        int count = 0;

        if (hasMinimumLength(pw)) {
            count++;
        }

        if (hasUppercase(pw)) {
```

```
            count++;
        }

        if (hasDigit(pw)) {
            count++;
        }

        if (hasSpecialCharacter(pw)) {
            count++;
        }

        if (count <= 1) {
            return "Weak";
        } else if (count <= 3) {
            return "Medium";
        } else {
            return "Strong";
        }
    }
}
```

Driver.java

```
public class Driver {

    public static void main(String[] args) {

        String[] passwords = {
            "abc",
            "abcdefgh",
            "Abcdefgh",
            "abcd1234",
            "Abcd1234!",
            "Password1"
        };

        for (String password : passwords) {

            System.out.println("Password: " + password);

            System.out.println(
                "Length >= 8: "
                + PasswordChecker.hasMinimumLength(password)
            );

            System.out.println(
                "Uppercase: "
                + PasswordChecker.hasUppercase(password)
            );

            System.out.println(
                "Digit: "
                + PasswordChecker.hasDigit(password)
            );

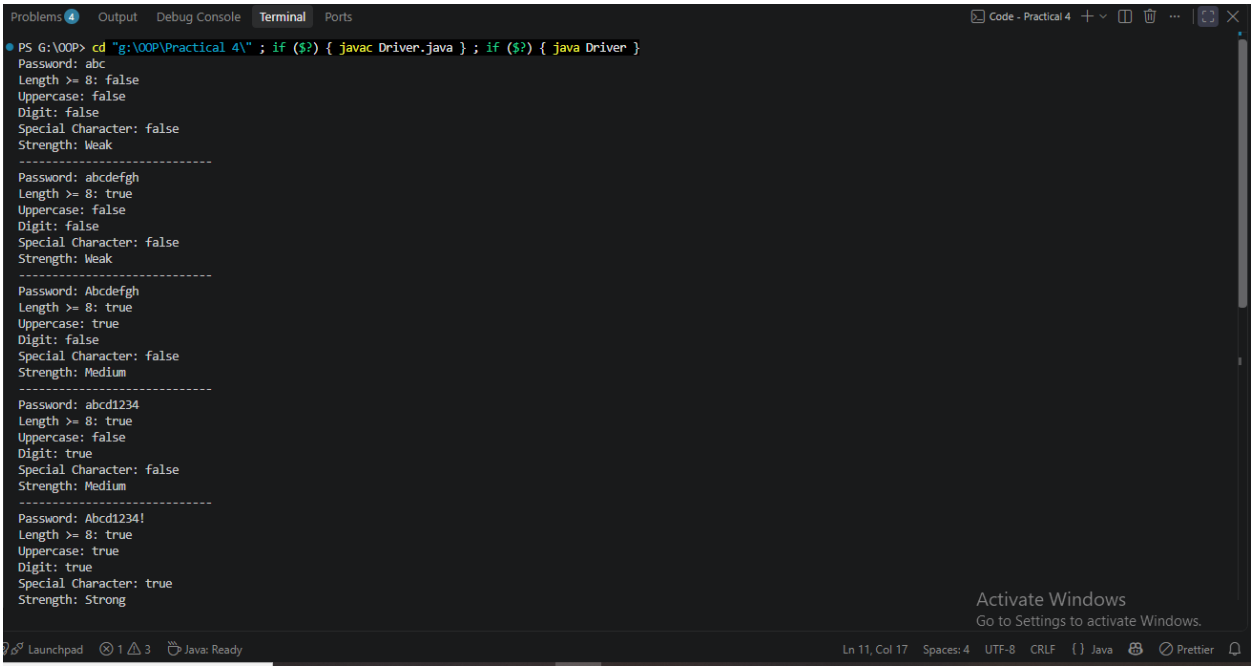
            System.out.println(
                "Special Character: "
```

```
                + PasswordChecker.hasSpecialCharacter(password)
            );

            System.out.println(
                "Strength: "
                + PasswordChecker.strength(password)
            );

            System.out.println("----------------------------
");
        }
    }
}
```

Output



2. Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like "10:05 alice Hello there" (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(" ", 3); skip lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive), increment a counter and append "time user: message" to a StringBuilder.

(e) Print "Matches: N" and the built report.

Expected output: keyword "hello" over 3 lines (1 malformed) → "Matches: 1" and the matching line.

Code:

ChatFilter.java

```
public class ChatFilter {

    public static boolean containsKeyword(String message,
String keyword) {
        return
message.toLowerCase().contains(keyword.toLowerCase());
    }



    public static String formatLine(String time, String user,
String message) {
        return time + " " + user + ": " + message;
    }
}
```

Driver.java

```
import java.util.Scanner;

public class Drive2{

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Chat log lines
        String[] logs = {
            "10:05 alice Hello there",
            "10:10 bob How are you?",
            "10:15 Charlie"    // Malformed line
        };
```

```
        // Read keyword
        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();

        int matches = 0;

        // StringBuilder for the final report
        StringBuilder report = new StringBuilder();

        // Process each log line
        for (String line : logs) {

            // Split into maximum 3 parts:
            // time, user, message
            String[] parts = line.split(" ", 3);

            // Skip malformed lines
            if (parts.length < 3) {
                continue;
            }

            String time = parts[0];
            String user = parts[1];
            String message = parts[2];

            // Check whether message contains keyword
            if (ChatFilter.containsKeyword(message, keyword)) {

                matches++;

                report.append(
                    ChatFilter.formatLine(time, user, message)
                );

                report.append("\n");
```

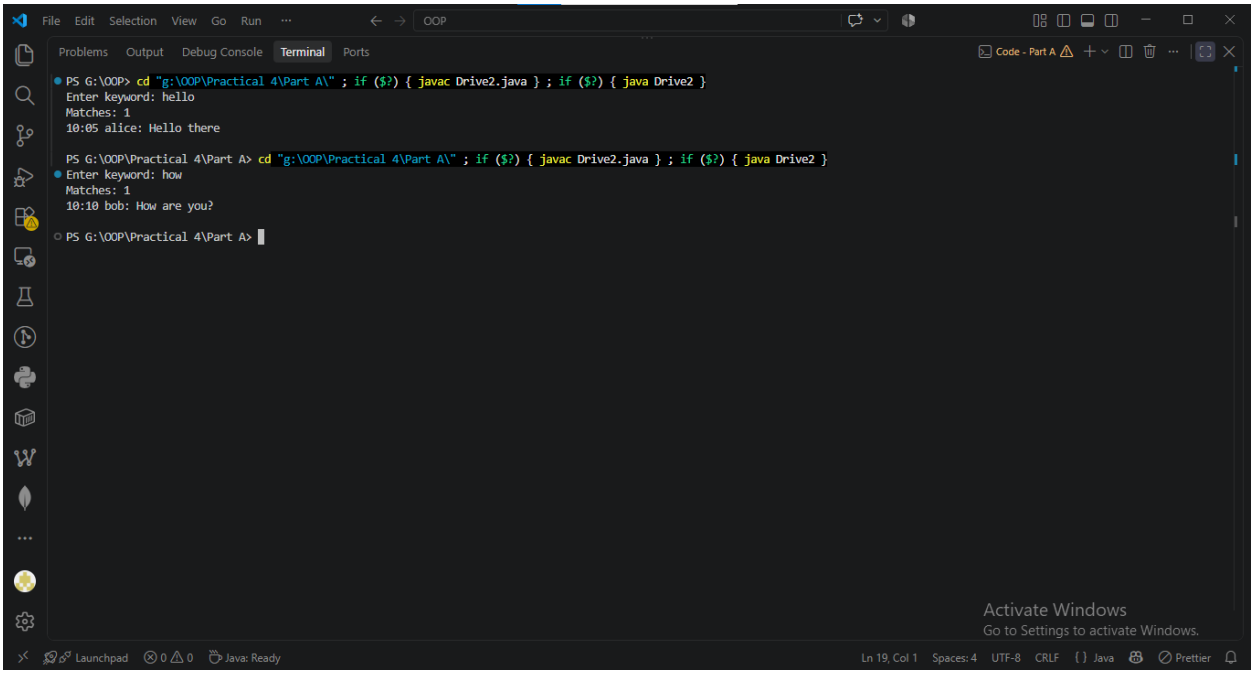
```
            }
        }

        // Print result
        System.out.println("Matches: " + matches);

        if (matches > 0) {
            System.out.println(report);
        }

        sc.close();
    }
}
```

Output



3. (For Advanced Learners – home) Template filler (TemplateFiller.java + Driver.java)
(a) In Driver.main, define a template like "Dear {name}, order {id} ships {date}." And two parallel arrays: names {"name", "id"} and values {"Riya", "A07"} (note {date} is not supplied).
(b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template.
(c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with "[?]" if not found.
(d) Print the filled template.
Expected output: the unsupplied {date} becomes [?].

Code:

TemplateFiller.java

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class TemplateFiller {

    public static String fillTemplate(
            String template,
            String[] names,
            String[] values) {

        // Pattern for placeholders like {name}, {id}, {date}
        Pattern pattern = Pattern.compile("\\{(\\w+)\\}");

        Matcher matcher = pattern.matcher(template);

        StringBuilder result = new StringBuilder();

        int lastEnd = 0;

        while (matcher.find()) {

            // Add text before the current placeholder
            result.append(template, lastEnd, matcher.start());

            // Get placeholder name without { }
            String placeholder = matcher.group(1);

            String replacement = "[?]";

            // Search for placeholder in names array
            for (int i = 0; i < names.length; i++) {
                if (names[i].equals(placeholder)) {
```

```
                    replacement = values[i];
                    break;
                }
            }

            // Add replacement
            result.append(replacement);

            lastEnd = matcher.end();
        }

        // Add remaining text after the last placeholder
        result.append(template, lastEnd, template.length());

        return result.toString();
    }
}
```

Driver.java

```
public class Driver1 {

    public static void main(String[] args) {

        String template =
                "Dear {name}, order {id} ships {date}.";

        String[] names = {
                "name",
                "id"
        };

        String[] values = {
                "Riya",
                "A07"
        };

        String result = TemplateFiller.fillTemplate(
                template,
                names,
                values
        );

        System.out.println(result);
    }
}
```

Output



Key Questions / Analysis / Interpretation to be evaluated during/after Implementation

1. What is the difference between String, StringBuilder and StringBuffer, and when is each preferred?

   Ans.

Type	Mutable?	Thread-safe?	Preferred when
`String`	✗ No	Yes, because immutable	Text that doesn't change
`StringBuilder`	✓ Yes	✗ No	Frequently modifying/combining text in a single thread
`StringBuffer`	✓ Yes	✓ Yes	Frequently modifying text where multiple threads may access it

`String`

`String` is **immutable**, meaning its contents cannot be changed after creation.

```
String name = "Rahul";
name = name + " Patel";
```

The original `"Rahul"` String isn't modified; a new String is created.

Use `String` for normal text:

```
String accountNumber = "AC0001";
```

StringBuilder

`StringBuilder` is **mutable**, so it can modify the same object without creating a new String for every operation.

```
StringBuilder statement = new StringBuilder();

statement.append("Account: AC0001\n");
statement.append("Balance: Rs.5000\n");
```

This is particularly useful for your **`StatementFormatter`**, where you're constructing a multi-line statement.

For normal single-threaded programs, `StringBuilder` is generally preferred over `StringBuffer` for repeated text modifications.

StringBuffer

`StringBuffer` is also mutable, but its methods are synchronized, making it suitable when the same buffer needs to be safely accessed by multiple threads.

```
StringBuffer buffer = new StringBuffer();

buffer.append("Hello");
buffer.append(" World");
```

The synchronization adds overhead, so you normally don't choose it unless thread safety is actually required.

2. What does the regular expression [6-9][0-9]{9} match, and why use anchors?

Ans.

The expression:

```
[6-9][0-9]{9}
```

matches a sequence consisting of:

- **one digit from 6 to 9**

- followed by **exactly 9 digits from 0 to 9**

So it represents a **10-digit number starting with 6, 7, 8, or 9**.

Example
`9876543210`

Breakdown:
`9 → [6-9]`
`876543210 → [0-9]{9}`

So it matches.

Another valid example:
`8123456789`

Why use anchors?

In your validator, we used:
`^[6-9][0-9]{9}$`

The anchors are:
`^`

means **start of the input**

and:
`$`

means **end of the input**.

Therefore:
`^[6-9][0-9]{9}$`

requires the **entire string** to match the pattern.

Without anchors, a longer string could contain a matching 10-digit sequence somewhere inside it.

3. Why should input be validated at the boundary before it is used?

Ans.

- **Prevents invalid data** from entering the system.
- Keeps the rest of the program simpler because it can rely on validated data.
- Reduces unexpected errors.
- Improves data consistency.
- Provides a clear place for input validation.
- Helps protect applications from malformed or malicious input.